



SQL PROJECT

-RAJAT GUSAIN
(FEB BATCH 2026)



TASK 1: DATA CLEANING & PREPARATION

1.1 IDENTIFY AND DELETE DUPLICATE ORDER_ID OR SHIPMENT_ID RECORDS

```
1 • SELECT * FROM fedex.fedex_orders;  
2 • select order_id ,count(*) as num_duplicates from fedex_orders group by order_id having count(*) >1;
```

<

Result Grid

Filter Rows:

Export: 

Wrap Cell Content: 

	order_id	num_duplicates
--	----------	----------------

: No duplicate record found in Order_Id

```

1 • SELECT * FROM fedex.fedex_shipments;
2 • select shipment_id ,count(*) as num_duplicates from fedex_shipments group by Shipment_ID having count(*) >1;

```

<

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



shipment_id	num_duplicates
-------------	----------------

: No duplicate record found in Shipment_Id



1.2 REPLACE NULL OR MISSING DELAY_HOURS VALUES IN THE SHIPMENTS TABLE WITH THE AVERAGE DELAY FOR THAT ROUTE_ID

customer_details SQL File 7* SQL File 8 fedex_delivery_agents fedex_orders fedex_routes **fedex_shipments** fedex_warehouses

Limit to 1000 rows

```
1 • SELECT * FROM fedex.fedex_shipments;
2 • select shipment_id ,count(*) as num_duplicates from fedex_shipments group by Shipment_ID having count(*) >1;
3
4
5 • select count(*) from fedex_shipments where Delay_Hours is null;
6
7
```

Result Grid Filter Rows: Export: Wrap Cell Content:

	count(*)
▶	0

: No NULL or missing Delay Hours Values found.



TASK 1.3 CONVERT ALL DATE COLUMNS (ORDER_DATE, PICKUP_DATE, DELIVERY_DATE) INTO YYYY-MM-DD HH:MM:SS FORMAT USING SQL DATE FUNCTIONS.

customer_details SQL File 7* SQL File 8 fedex_delivery_agents **fedex_orders** fedex_routes fedex_shipments fedex_warehou

Limit to 1000 rows

```
1 • SELECT * FROM fedex.fedex_orders;
2 • select order_id ,count(*) as num_duplicates from fedex_orders group by order_id having count(*) >1;
3 • describe fedex_orders;
4
```

<

Result Grid Filter Rows: Export: Wrap Cell Content:

	Field	Type	Null	Key	Default	Extra
►	Order_ID	text	YES		NULL	
	Customer_ID	text	YES		NULL	
	Order_Date	datetime	YES		NULL	
	Route_ID	text	YES		NULL	
	Warehouse_ID	text	YES		NULL	
	Order_Amount	double	YES		NULL	
	Delivery_Type	text	YES		NULL	
			YES		NULL	

Result 4

: Data type of 'Order_Date' column already changed to 'DATETIME' while importing the 'fedex_orders' table.

6

7

8

9 • `describe fedex_shipments;`

<

Result Grid



Filter Rows:

Export:



Wrap Cell Content:

IA

	Field	Type	Null	Key	Default	Extra
	Route_ID	text	YES		NULL	
	Warehouse_ID	text	YES		NULL	
	Pickup_Date	datetime	YES		NULL	
	Delivery_Date	datetime	YES		NULL	
	Delivery_Status	text	YES		NULL	
	Delay_Hours	double	YES		NULL	
	Delivery_Feedback	text	YES		NULL	
					NULL	

: Data type of 'Pickup_Date' & 'Delivery_Date' column already changed to 'DATE' while importing the fedex_shipments table.

```
5
6
7 SELECT Shipment_ID, Pickup_Date, Delivery_Date,
8     CASE
9         WHEN Delivery_Date < Pickup_Date THEN '⚠ ERROR: Delivery Before Pickup'
10        WHEN Delivery_Date = Pickup_Date THEN 'Same Day Delivery'
11        ELSE 'Valid'
12    END AS Quality_Check_Flag
13 FROM fedex_shipments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	Shipment_ID	Pickup_Date	Delivery_Date	Quality_Check_Flag
▶	S00001	2024-08-14 12:12:39	2024-08-15 10:12:46	Valid
	S00002	2024-11-12 11:38:21	2024-11-13 13:55:29	Valid
	S00003	2025-04-23 15:13:08	2025-04-24 13:04:57	Valid
	S00004	2024-09-06 04:34:17	2024-09-07 06:55:16	Valid
	S00005	2025-08-11 13:49:33	2025-08-12 06:29:25	Valid
	S00006	2025-04-17 03:13:45	2025-04-18 00:11:32	Valid
	S00007	2024-09-29 21:28:17	2024-09-30 12:47:50	Valid
	S00008	2024-12-18 18:04:14	2024-12-19 07:44:40	Valid

Result 15

```
5 • SELECT o.Order_ID, o.Route_ID AS Order_Route, o.Warehouse_ID AS Order_Warehouse
6 FROM fedex_orders o
7 LEFT JOIN fedex_routes r ON o.Route_ID = r.Route_ID
8 LEFT JOIN fedex_warehouses w ON o.Warehouse_ID = w.Warehouse_ID
9 WHERE r.Route_ID IS NULL OR w.Warehouse_ID IS NULL;
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



Order_ID

Order_Route

Order_Warehouse

: Referential Integrity between ORDERS , ROUTES and WAREHOUSE


```
16 • SELECT s.Shipment_ID, s.Order_ID  
17 FROM fedex_shipments s  
18 LEFT JOIN fedex_orders o ON s.Order_ID = o.Order_ID  
19 WHERE o.Order_ID IS NULL;  
20  
21
```



Result Grid



Filter Rows:

Export:



Wrap Cell Content:



Shipment_ID

Order_ID

: Referential Integrity between ORDERS and SHIPMENTS.



TASK 2: DELIVERY DELAY ANALYSIS

2.1 CALCULATE DELIVERY DELAY (IN HOURS) FOR EACH SHIPMENT USING DELIVERY_DATE – PICKUP_DATE.

```
22 • select Shipment_ID, Order_ID , Pickup_Date , Delivery_Date ,timestampdiff(HOUR,Pickup_Date,Delivery_Date) as  
23 Delivery_Delay_Hours from fedex_shipments;
```

< **Result Grid** | | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	Shipment_ID	Order_ID	Pickup_Date	Delivery_Date	Delivery_Delay_Hours
▶	S00001	O0289	2024-08-14 12:12:39	2024-08-15 10:12:46	22
	S00002	O0166	2024-11-12 11:38:21	2024-11-13 13:55:29	26
	S00003	O0156	2025-04-23 15:13:08	2025-04-24 13:04:57	21
	S00004	O0237	2024-09-06 04:34:17	2024-09-07 06:55:16	26
	S00005	O0047	2025-08-11 13:49:33	2025-08-12 06:29:25	16
	S00006	O0022	2025-04-17 03:13:45	2025-04-18 00:11:32	20
	S00007	O0078	2024-09-29 21:28:17	2024-09-30 12:47:50	15
	S00008	O0089	2024-12-10 10:04:14	2024-12-11 07:44:49	21
	S00009	O0082	2024-04-27 17:37:20	2024-05-01 20:51:10	99
	S00010	O0027	2024-03-08 17:26:59	2024-03-09 18:43:20	25
	S00011	O0198	2025-06-17 11:51:28	2025-06-18 05:20:46	17
	S00012	O0073	2024-06-12 13:37:06	2024-06-14 08:00:17	42
	S00013	O0044	2024-03-06 20:17:41	2024-03-09 21:27:16	73
	S00014	O0270	2024-10-11 07:34:27	2024-10-12 10:02:35	26
	S00015	O0093	2024-12-03 18:23:30	2024-12-14 15:15:37	260
	S00016	O0080	2024-01-09 07:47:47	2024-01-12 16:13:41	80
	S00017	O0270	2024-10-11 04:39:20	2024-10-12 02:39:45	22
	S00018	O0084	2024-05-15 17:27:28	2024-05-16 18:17:49	24
	S00019	O0187	2024-10-26 20:40:02	2024-10-28 17:47:02	45
	S00020	O0230	2025-07-10 18:22:59	2025-07-17 10:55:01	160



2.2 FIND THE TOP 10 DELAYED ROUTES BASED ON AVERAGE DELAY HOURS.

```
4 -- top 10 delayed routes based on average delay hours.
5 • SELECT r.Route_ID, r.Source_City, r.Destination_City,
6 ROUND(AVG(s.Delay_Hours), 2) AS Average_Delay_Hours, COUNT(s.Shipment_ID) AS Total_Shipments
7 FROM fedex_routes r
8 JOIN fedex_shipments s ON r.Route_ID = s.Route_ID
9 GROUP BY r.Route_ID, r.Source_City, r.Destination_City
10 ORDER BY Average_Delay_Hours DESC
11 LIMIT 10;
```

Result Grid

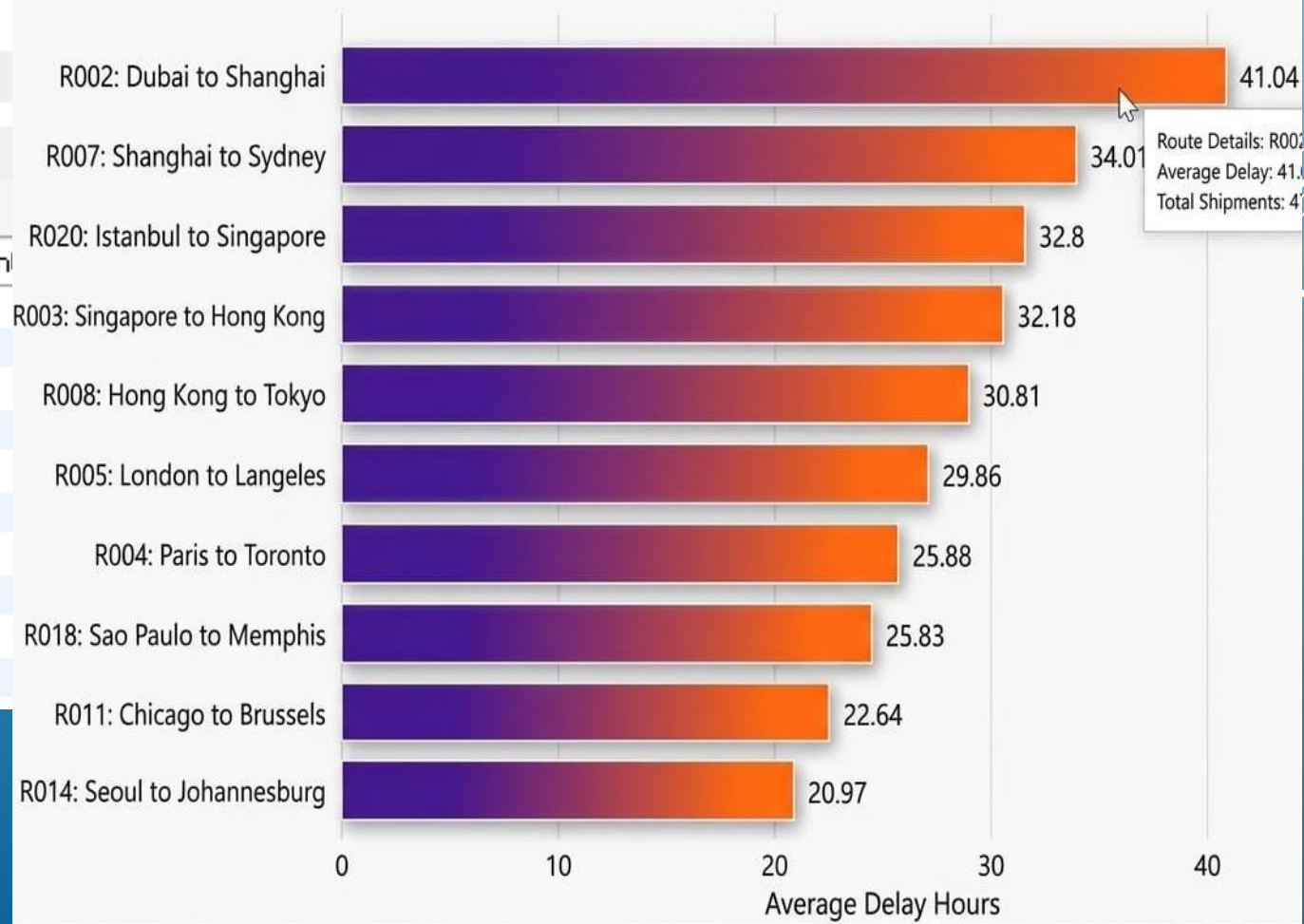
Filter Rows:

Export: Wrap Cell Content: ☐

	Route_ID	Source_City	Destination_City	Average_Delay_Hours	Total_Shipments
▶	R002	Dubai	Shanghai	41.04	47
	R007	Shanghai	Sydney	34.01	51
	R020	Istanbul	Singapore	32.8	42
	R003	Singapore	Hong Kong	32.18	34
	R008	Hong Kong	Tokyo	30.81	44
	R005	London	Los Angeles	29.86	48
	R004	Paris	Toronto	25.88	49
	R018	Sao Paulo	Memphis	25.83	59
	R011	Chicago	Brussels	22.64	48
	R014	Seoul	Johannesburg	20.97	53



TOP 10 DELAYED FEDEX ROUTES (by Avg. Delay Hours)





2.3 USE SQL WINDOW FUNCTIONS TO RANK SHIPMENTS BY DELAY WITHIN EACH WAREHOUSE_ID

```
25      -- Using SQL window functions to rank shipments by delay within each Warehouse_ID.
26 •    select Warehouse_ID , Shipment_ID , Delay_Hours,
27      rank() over(partition by Warehouse_ID order by Delay_Hours desc)
28      as Delay_Rank_In_Warehouse from fedex_shipments order by Warehouse_ID , Delay_Rank_In_Warehouse;
29
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	Warehouse_ID	Shipment_ID	Delay_Hours	Delay_Rank_In_Warehouse
	W001	S00648	205.7	1
	W001	S00670	194.6	2
	W001	S00997	193.7	3
	W001	S00349	171.6	4
	W001	S00069	170.8	5
	W001	S00148	138.7	6
	W001	S00192	99.5	7
	W001	S00153	95.4	8
	W001	S00739	71.4	9
	W001	S00816	66.3	10
	W001	S00550	64.5	11
	W001	S00393	63.9	12
	W001	S00299	59.2	13
	W001	S00376	57.6	14
	W001	S00027	57.2	15
	W001	S00590	46.7	16
	W001	S00874	33.4	17
	W001	S00169	31.4	18
	W001	S00669	29.7	19
	W001	S00994	27.4	20



2.4 IDENTIFY THE AVERAGE DELAY PER DELIVERY_TYPE (EXPRESS / STANDARD) TO COMPARE SERVICE-LEVEL EFFICIENCY.

```
11  -- Identifying the average delay per Delivery_Type (Express / Standard) to compare service-level efficiency.
12  • select o.Delivery_Type, round(avg(s.delay_hours),2) as Avg_Delay_Hours, count(s.shipment_id) as Total_Shipment
13  FROM fedex_orders o
14  JOIN fedex_shipments s ON o.Order_ID = s.Order_ID
15  GROUP BY o.Delivery_Type
16  ORDER BY Avg_Delay_Hours ASC;
```



FedEx Shipment Count and Avg. Delay by Delivery Type

Express: 337



- Standard (663 Count | Avg. Delay: 20.85 hrs)
- Express (337 Count | Avg. Delay: 22.50 hrs)

Standard: 663

Based on recent Fedex_Order
and Fedex Shipment data

Result Grid



Filter Rows:

Export:

	Delivery_Type	Avg_Delay_Hours	Total_Shipment
▶	Standard	20.85	663
	Express	22.5	337

TASK 3: ROUTE OPTIMIZATION INSIGHTS

3.1 AVERAGE TRANSIT TIME (IN HOURS) ACROSS ALL SHIPMENTS.

```

14  -- Average transit time (in hours) across all shipments.
15  •  SELECT r.Route_ID, r.Source_City, r.Destination_City,
16  ROUND(AVG(TIMESTAMPDIFF(SECOND, s.Pickup_Date, s.Delivery_Date) / 3600.0), 2) AS Avg_Actual_Hours,
17  r.Avg_Transit_Time_Hours AS Planned_Hours,
18  ROUND(AVG(TIMESTAMPDIFF(SECOND, s.Pickup_Date, s.Delivery_Date) / 3600.0) - r.Avg_Transit_Time_Hours, 2)
19  AS Variance_Hours FROM fedex_routes r
20  JOIN fedex_shipments s ON r.Route_ID = s.Route_ID WHERE s.Delivery_Status = 'Delivered'
21  GROUP BY r.Route_ID, r.Source_City, r.Destination_City, r.Avg_Transit_Time_Hours ORDER BY Variance_Hours DESC;

```

Route_ID	Source_City	Destination_City	Avg_Actual_Hours	Planned_Hours	Variance_Hours
R002	Dubai	Shanghai	64.89	18.5	46.39
R004	Paris	Toronto	86.10	49.1	37
R020	Istanbul	Singapore	53.94	17.2	36.74
R007	Shanghai	Sydney	67.02	30.5	36.52
R008	Hong Kong	Tokyo	53.19	17.1	36.09
R005	London	Los Angeles	57.40	23.3	34.1
R003	Singapore	Hong Kong	57.58	23.5	34.08
R018	Sao Paulo	Memphis	72.90	43	29.9
R014	Seoul	Johannesburg	54.43	27.3	27.13
R011	Chicago	Brussels	42.03	15.5	26.53
R010	Los Angeles	Amsterdam	34.04	11.6	22.44
R017	Madrid	Mexico City	67.14	45.1	22.04
R012	Sydney	Madrid	63.92	42.3	21.62
R019	Tokyo	Singapore	52.02	21.2	21.52

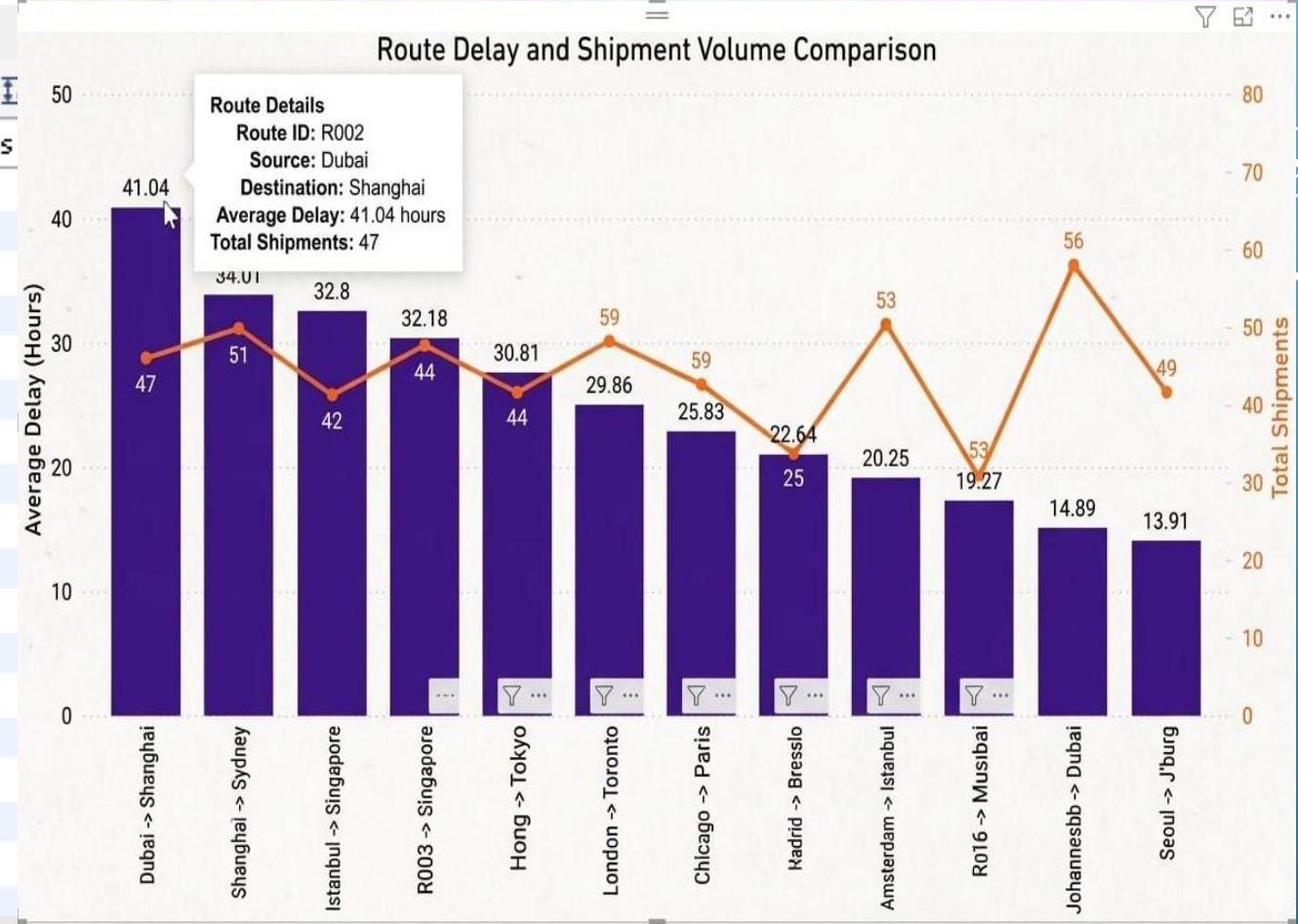

```

24 -- Average delay (in hours) per route.
25 • SELECT r.Route_ID, r.Source_City,r.Destination_City,
26 ROUND(AVG(s.Delay_Hours), 2) AS Average_Delay_Hours,
27 COUNT(s.Shipment_ID) AS Total_Shipments FROM fedex_routes r JOIN fedex_shipments s ON r.Route_ID = s.Route_ID
28 GROUP BY r.Route_ID, r.Source_City, r.Destination_City
29 ORDER BY Average_Delay_Hours DESC;

```

FedEx 3.2: AVERAGE DELAY (IN HOURS) PER ROUTE

Route_ID	Source_City	Destination_City	Average_Delay_Hours	Total_Shipments
R002	Dubai	Shanghai	41.04	47
R007	Shanghai	Sydney	34.01	51
R020	Istanbul	Singapore	32.8	42
R003	Singapore	Hong Kong	32.18	34
R008	Hong Kong	Tokyo	30.81	44
R005	London	Los Angeles	29.86	48
R004	Paris	Toronto	25.88	49
R018	Sao Paulo	Memphis	25.83	59
R011	Chicago	Brussels	22.64	48
R014	Seoul	Johannesburg	20.97	53
R010	Los Angeles	Amsterdam	20.25	52
R017	Madrid	Mexico City	19.27	53
R015	Amsterdam	Istanbul	15.19	56
R016	Brussels	Mumbai	14.89	71
R006	Frankfurt	Chicago	14.01	56
R019	Johannesb...	Dubai	13.91	49
R012	Sydney	Madrid	13.89	56
R013	Tokyo	Sao Paulo	11.95	50





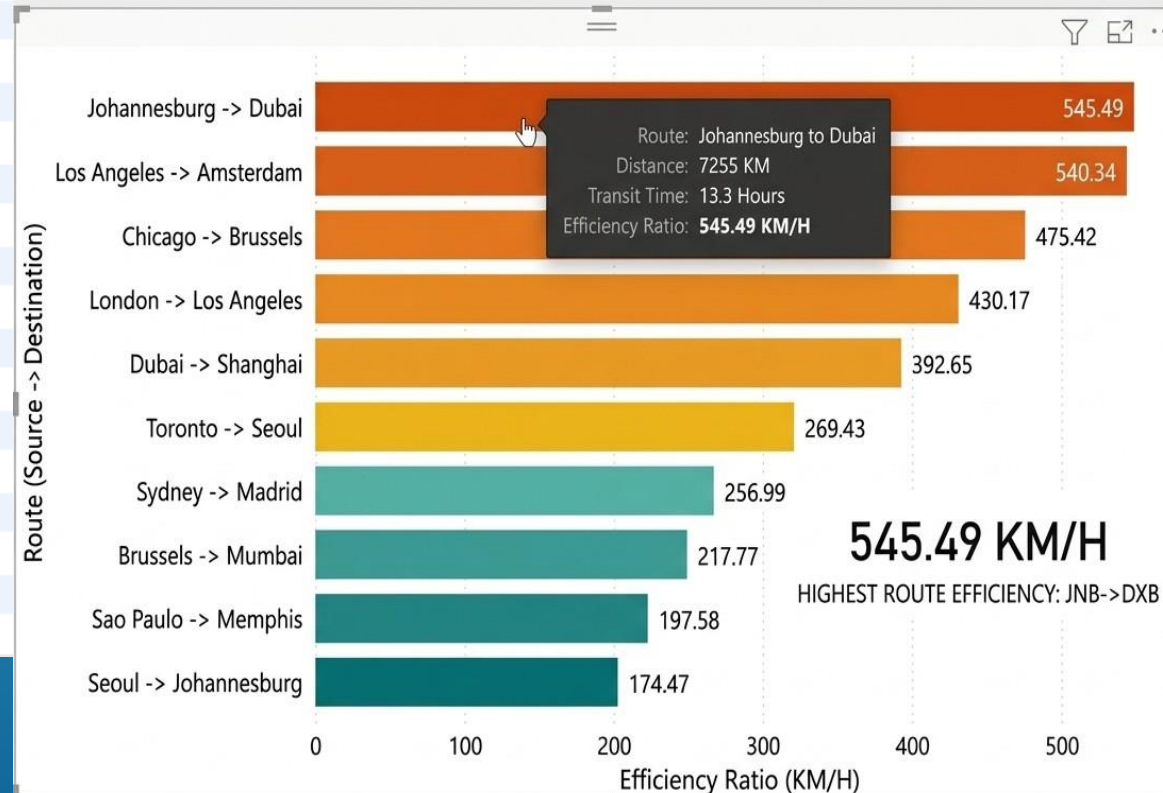
3.3 DISTANCE-TO-TIME EFFICIENCY RATIO = DISTANCE_KM / AVG TRANSIT TIME HOURS.

```
32 -- Distance-to-time efficiency ratio = Distance_KM / Avg_Transit_Time_Hours.  
33 • select Route_ID, Source_City, Destination_City, Distance_KM, Avg_Transit_Time_Hours ,  
34 round(Distance_KM/Avg_Transit_Time_Hours ,2) as Efficiency_Ratio_KMH  
35 from fedex_routes order by Efficiency_Ratio_KMH desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Route_ID	Source_City	Destination_City	Distance_KM	Avg_Transit_Time_Hours	Efficiency_Ratio_KMH
R019	Johannesburg	Dubai	7255	13.3	545.49
R010	Los Angeles	Amsterdam	6268	11.6	540.34
R011	Chicago	Brussels	7369	15.5	475.42
R005	London	Los Angeles	10023	23.3	430.17
R002	Dubai	Shanghai	7264	18.5	392.65
R009	Toronto	Seoul	5582	15.5	360.13
R012	Sydney	Madrid	11397	42.3	269.43
R016	Brussels	Mumbai	8018	31.2	256.99
R018	Sao Paulo	Memphis	9347	43	217.37
R014	Seoul	Johannesburg	5394	27.3	197.58
R004	Paris	Toronto	8555	49.1	174.24
R007	Shanghai	Sydney	5297	30.5	173.67
R017	Madrid	Mexico City	6651	45.1	147.47
R020	Istanbul	Singapore	2513	17.2	146.1
R001	Memphis	Frankfurt	4620	32.1	143.93
R013	Tokyo	Sao Paulo	3794	31.3	121.21
R008	Hong Kong	Tokyo	1846	17.1	107.95
R006	Frankfurt	Chicago	2364	23.3	101.46
R015	Amsterdam	Istanbul	606	11.5	52.7

3.3: DISTANCE-TO-TIME EFFICIENCY RATIO = DISTANCE_KM / AVG_TRANSIT_TIME_HOURS
FedExRoutes Efficiency Analysis





3.4 IDENTIFY 3 ROUTES WITH THE WORST EFFICIENCY RATIO (LOWEST DISTANCE-TO-TIME).

```
38 -- Identifying 3 routes with the worst efficiency ratio (lowest distance-to-time).
39 • select Route_ID, Source_City, Destination_City, Distance_KM, Avg_Transit_Time_Hours ,
40 round(Distance_KM/Avg_Transit_Time_Hours ,2) as Efficiency_Ratio_KMH
41 from fedex_routes order by Efficiency_Ratio_KMH asc limit 3;
```

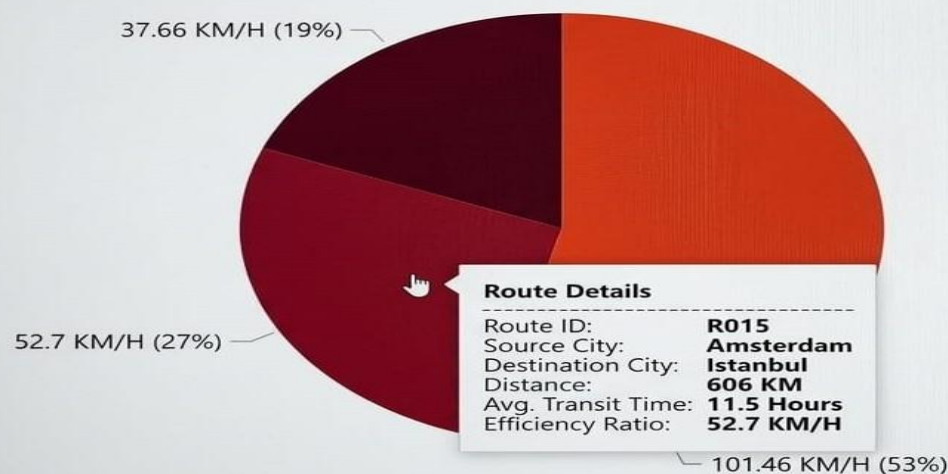
Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

Route_ID	Source_City	Destination_City	Distance_KM	Avg_Transit_Time_Hours	Efficiency_Ratio_KMH
R003	Singapore	Hong Kong	885	23.5	37.66
R015	Amsterdam	Istanbul	606	11.5	52.7
R006	Frankfurt	Chicago	2364	23.3	101.46



3.4 TOP 3 WORST EFFICIENCY ROUTES (Ratio KM/H)

Lowest Distance-to-Time Efficiency Ratio



- R003: Singapore to Hong Kong
- R015: Amsterdam to Istanbul
- R006: Frankfurt to Chicago



3.5 FIND ROUTES WITH >20% OF SHIPMENTS DELAYED BEYOND EXPECTED TRANSIT TIME.

```
47 -- Finding routes with >20% of shipments delayed beyond expected transit time.
48 • SELECT r.Route_ID, r.Source_City, r.Destination_City, COUNT(s.Shipment_ID) AS Total_Shipments,
49 SUM(CASE WHEN (TIMESTAMPDIFF(SECOND, s.Pickup_Date, s.Delivery_Date) / 3600.0) > r.Avg_Transit_Time_Hours
50 THEN 1 ELSE 0 END) AS Delayed_Shipment_Count,
51 ROUND((SUM(CASE WHEN (TIMESTAMPDIFF(SECOND, s.Pickup_Date, s.Delivery_Date) / 3600.0) > r.Avg_Transit_Time_Hours
52 THEN 1 ELSE 0 END) * 100.0) / COUNT(s.Shipment_ID), 2) AS Delay_Percentage
53 FROM fedex_routes r JOIN fedex_shipments s ON r.Route_ID = s.Route_ID WHERE s.Delivery_Status = 'Delivered'
54 GROUP BY r.Route_ID, r.Source_City, r.Destination_City HAVING Delay_Percentage > 20
55 ORDER BY Delay_Percentage desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Route_ID	Source_City	Destination_City	Total_Shipments	Delayed_Shipment_Count	Delay_Percentage
R019	Johannesburg	Dubai	45	45	100.00
R012	Sydney	Madrid	50	50	100.00
R010	Los Angeles	Amsterdam	47	46	97.87
R007	Shanghai	Sydney	46	45	97.83
R005	London	Los Angeles	45	44	97.78
R008	Hong Kong	Tokyo	41	40	97.56
R002	Dubai	Shanghai	40	39	97.50
R015	Amsterdam	Istanbul	51	49	96.08
R013	Tokyo	Sao Paulo	41	39	95.12
R020	Istanbul	Singapore	35	33	94.29
R009	Toronto	Seoul	34	32	94.12
R006	Frankfurt	Chicago	49	46	93.88
R018	Sao Paulo	Memphis	48	45	93.75
R011	Chicago	Brussels	47	44	93.62

```
66  -- Recommend potential routes or hub pairs for optimization.
67  •  SELECT r.Source_City, r.Destination_City, COUNT(s.Shipment_ID) AS Shipment_Volume,
68     ROUND(AVG(s.Delay_Hours), 2) AS Avg_Delay,
69     CASE WHEN COUNT(s.Shipment_ID) > 50 AND AVG(s.Delay_Hours) < 12 THEN 'High Priority: Establish Direct Route'
70     WHEN COUNT(s.Shipment_ID) > 50 THEN 'Optimal: High Volume Efficiency'
71     ELSE 'Monitor' END AS Recommendation
72  FROM fedex_routes r JOIN fedex_shipments s ON r.Route_ID = s.Route_ID
73  GROUP BY r.Source_City, r.Destination_City ORDER BY Shipment_Volume DESC;
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	Source_City	Destination_City	Shipment_Volume	Avg_Delay	Recommendation
	Brussels	Mumbai	71	14.89	Optimal: High Volume Efficiency
	Sao Paulo	Memphis	59	25.83	Optimal: High Volume Efficiency
	Amsterdam	Istanbul	56	15.19	Optimal: High Volume Efficiency
	Sydney	Madrid	56	13.89	Optimal: High Volume Efficiency
	Frankfurt	Chicago	56	14.01	Optimal: High Volume Efficiency
	Seoul	Johannesburg	53	20.97	Optimal: High Volume Efficiency
	Madrid	Mexico City	53	19.27	Optimal: High Volume Efficiency
	Los Angeles	Amsterdam	52	20.25	Optimal: High Volume Efficiency
	Shanghai	Sydney	51	34.01	Optimal: High Volume Efficiency
	Tokyo	Sao Paulo	50	11.95	Monitor

-High-Volume "Bypass" Candidates:

These are the Routes with very high shipment volumes that also suffer from consistent delays are the strongest candidates for Point-to-Point routing. Instead of moving these through a central hub, a direct line can be established to reduce handling time.

```

76      -- Distance Vs Time
77      • SELECT Source_City, Destination_City, Distance_KM,
78      ROUND(Distance_KM / Avg_Transit_Time_Hours, 2) AS KM_per_Hour,
79      'Potential for Mid-Way Sorting Hub' AS Optimization_Strategy
80      FROM fedex_routes WHERE (Distance_KM / Avg_Transit_Time_Hours) < 60
81      AND Distance_KM > 600 ORDER BY KM_per_Hour ASC;

```

< Result Grid Filter Rows: Export: Wrap Cell Content:					
	Source_City	Destination_City	Distance_KM	KM_per_Hour	Optimization_Strategy
▶	Singapore	Hong Kong	885	37.66	Potential for Mid-Way Sorting Hub
	Amsterdam	Istanbul	606	52.7	Potential for Mid-Way Sorting Hub

-The "Efficiency Gap" (Distance vs. Time)

If a route covers a long distance but has a very low average speed (Efficiency Ratio), it suggests the route is winding, has too many stops, or faces infrastructure bottlenecks. These are candidates for an Intermediate Sorting Hub.



TASK 4: WAREHOUSE PERFORMANCE

4.1 FIND THE TOP 3 WAREHOUSES WITH THE HIGHEST AVERAGE DELAY IN SHIPMENTS DISPATCHED.

```
3 -- Find the top 3 warehouses with the highest average delay in shipments dispatched.
4 • select w.Warehouse_ID, w.city, round(avg(s.Delay_Hours),2) as Avg_Delay_Hours,
5 count(s.Shipment_ID) as Total_Dispatched from fedex_warehouses w join fedex_shipments s
6 on w.Warehouse_ID=s.Warehouse_ID group by w.Warehouse_ID,w.city having Total_Dispatched>=10
7 order by Avg_Delay_Hours desc limit 3;
```

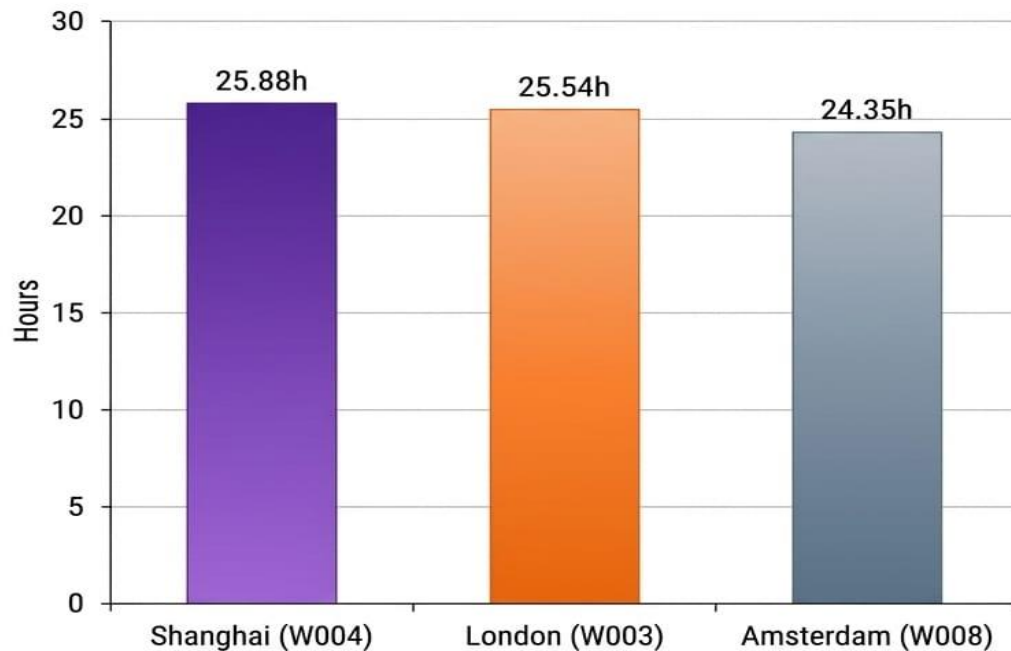
Result Grid | Filter Rows: | Export: | Wr.

	Warehouse_ID	city	Avg_Delay_Hours	Total_Dispatched
▶	W004	Shanghai	25.88	90
	W003	London	25.54	107
	W008	Amsterdam	24.35	112

TASK 4: WAREHOUSE PERFORMANCE - TOP 3 DELAYED DISPATCH ANALYSIS



TOP 3 WAREHOUSES BY AVG. DISPATCH DELAY (HOURS)



PERFORMANCE DATA SUMMARY

Warehouse ID	City	Avg. Delay (hrs)	Total Dispatched
W004	Shanghai	25.88	90
W003	London	25.54	107
W008	Amsterdam	24.35	112

TOTAL DISPATCHED VOLUME

Shanghai (W004) → **90** Shipments

London (W003) → **107** Shipments

Amsterdam (W008) → **112** Shipments

Data derived from latest FedEx warehouse & shipment records. Analysis focuses on top 3 delayed warehouses with Total_Dispatched >= 10.



4.2 CALCULATE TOTAL SHIPMENTS VS DELAYED SHIPMENTS FOR EACH WAREHOUSE.

```
10      -- Calculate total shipments vs delayed shipments for each warehouse.
11      • SELECT w.Warehouse_ID, w.city, COUNT(s.Shipment_ID) AS Total_Shipments,
12          SUM(CASE WHEN s.Delay_Hours > 0 THEN 1 ELSE 0 END) AS Delayed_Shipments,
13          ROUND((SUM(CASE WHEN s.Delay_Hours > 0 THEN 1 ELSE 0 END) * 100.0) / COUNT(s.Shipment_ID), 2)
14          AS Delay_Rate_Percentage FROM fedex_shipments s JOIN fedex_warehouses w ON s.Warehouse_ID = w.Warehouse_ID
15      GROUP BY w.Warehouse_ID, w.city
16      ORDER BY Delay_Rate_Percentage DESC;
17
```

<

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	Warehouse_ID	city	Total_Shipments	Delayed_Shipments	Delay_Rate_Percentage
▶	W007	Tokyo	96	94	97.92
	W006	Chicago	99	94	94.95
	W004	Shanghai	90	85	94.44
	W002	Singapore	97	91	93.81
	W003	London	107	100	93.46
	W001	Memphis	103	96	93.20
	W009	Madrid	91	84	92.31
	W005	Toronto	102	94	92.16
	W010	Johannesburg	103	94	91.26
	W008	Amsterdam	112	100	89.29

4.3 USE CITES TO IDENTIFY WAREHOUSES WHERE AVERAGE DELAY EXCEEDS THE GLOBAL AVERAGE DELAY.

```
19  -- Use CTEs to identify warehouses where average delay exceeds the global average delay.
20  WITH
21  -- CTE 1
22  Global_Metrics AS (SELECT AVG(Delay_Hours) AS global_avg FROM fedex_shipments),
23  -- CTE 2
24  Warehouse_Metrics AS (SELECT w.Warehouse_ID, w.city, AVG(s.Delay_Hours) AS wh_avg, COUNT(s.Shipment_ID) AS total_volume
25  FROM fedex_shipments s JOIN fedex_warehouses w ON s.Warehouse_ID = w.Warehouse_ID
26  GROUP BY w.Warehouse_ID, w.city)
27  SELECT wm.Warehouse_ID, wm.city, wm.total_volume AS Total_Shipments,
28  ROUND(wm.wh_avg, 2) AS Warehouse_Avg_Delay, ROUND(gm.global_avg, 2) AS Global_Avg_Delay,
29  ROUND(wm.wh_avg - gm.global_avg, 2) AS Hours_Above_Global FROM Warehouse_Metrics wm
30  CROSS JOIN Global_Metrics gm WHERE wm.wh_avg > gm.global_avg ORDER BY Hours_Above_Global DESC;
```

Result Grid						
Filter Rows:		Export:		Wrap Cell Content:		
	Warehouse_ID	city	Total_Shipments	Warehouse_Avg_Delay	Global_Avg_Delay	Hours_Above_Global
▶	W004	Shanghai	90	25.88	21.41	4.47
	W003	London	107	25.54	21.41	4.14
	W008	Amsterdam	112	24.35	21.41	2.94
	W005	Toronto	102	22.43	21.41	1.03
	W007	Tokyo	96	22.06	21.41	0.65
	W010	Johannesburg	103	21.8	21.41	0.4



Result Grid



Form Editor


```
33 -- Rank all warehouses based on on-time delivery percentage.
34 -- CTE
35 WITH Facility_Performance AS (SELECT w.Warehouse_ID, w.city, COUNT(s.Shipment_ID) AS Total_Shipments,
36 SUM(CASE WHEN s.Delay_Hours = 0 THEN 1 ELSE 0 END) AS On_Time_Count,
37 ROUND((SUM(CASE WHEN s.Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0) / COUNT(s.Shipment_ID), 2) AS On_Time_Percentage
38 FROM fedex_shipments s JOIN fedex_warehouses w ON s.Warehouse_ID = w.Warehouse_ID
39 GROUP BY w.Warehouse_ID, w.city HAVING Total_Shipments > 20)
40 -- Ranking
41 SELECT Warehouse_ID, city, Total_Shipments, On_Time_Percentage,
42 DENSE_RANK() OVER (ORDER BY On_Time_Percentage DESC) AS Performance_Rank FROM Facility_Performance
43 ORDER BY Performance_Rank;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Warehouse_ID	city	Total_Shipments	On_Time_Percentage	Performance_Rank
W008	Amsterdam	112	10.71	1
W010	Johannesburg	103	8.74	2
W005	Toronto	102	7.84	3
W009	Madrid	91	7.69	4
W001	Memphis	103	6.80	5
W003	London	107	6.54	6
W002	Singapore	97	6.19	7
W004	Shanghai	90	5.56	8
W006	Chicago	99	5.05	9
W007	Tokyo	96	2.08	10



TASK 5: DELIVERY AGENT PERFORMANCE

5.1 RANK DELIVERY AGENTS (PER ROUTE) BY ON-TIME DELIVERY PERCENTAGE.

```
4  -- Ranking delivery agents (per route) by on-time delivery percentage.
5  -- CTE
6  WITH Agent_Performance AS (SELECT Route_ID, Agent_ID, COUNT(Shipment_ID) AS Total_Shipments,
7  SUM(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END) AS On_Time_Count,
8  ROUND((SUM(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0) / COUNT(Shipment_ID), 2) AS On_Time_Percentage
9  FROM fedex_shipments GROUP BY Route_ID, Agent_ID)
10 SELECT Route_ID, Agent_ID, Total_Shipments, On_Time_Percentage, DENSE_RANK() OVER (PARTITION BY Route_ID
11 ORDER BY On_Time_Percentage DESC) AS Agent_Rank_On_Route FROM Agent_Performance
12 ORDER BY Route_ID, Agent_Rank_On_Route;
```

Result Grid					
Filter Rows:		Export:		Wrap Cell Content:	
Route_ID	Agent_ID	Total_Shipments	On_Time_Percentage	Agent_Rank_On_Route	
R001	A0014	1	100.00	1	
R001	A0009	1	100.00	1	
R001	A0019	2	50.00	2	
R001	A0042	2	50.00	2	
R001	A0001	3	33.33	3	
R001	A0038	1	0.00	4	
R001	A0033	1	0.00	4	
R001	A0008	2	0.00	4	
R001	A0025	2	0.00	4	
R001	A0018	1	0.00	4	

Output				
Action Output				
#	Time	Action	Message	
17	12:57:55	WITH Agent_Performance AS (SELECT Route_ID, Agent_ID, COUNT(Shipment_ID) AS Tot...	644 row(s) returned	
18	12:58:20	WITH Agent_Performance AS (SELECT Route_ID, Agent_ID, COUNT(Shipment_ID) AS Tot...	644 row(s) returned	



5.2 FIND AGENTS WHOSE ON-TIME % IS BELOW 85%.

```
15 -- Find agents whose on-time % is below 85%.
16 SELECT Agent_ID,count(shipment_id) as Total_Shipments ,sum(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END)
17 AS On_Time_Count,ROUND((SUM(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0) / COUNT(Shipment_ID), 2)
18 AS On_Time_Percentage FROM fedex_shipments group by agent_id having On_Time_Percentage <85.00
19 ORDER BY On_Time_Percentage desc;
```

Result Grid Filter Rows: Export: Wrap Cell Content:

	Agent_ID	Total_Shipments	On_Time_Count	On_Time_Percentage
▶	A0014	21	5	23.81
	A0019	23	5	21.74
	A0002	10	2	20.00
	A0043	16	3	18.75
	A0009	23	4	17.39
	A0018	26	4	15.38
	A0028	14	2	14.29
	A0042	22	3	13.64
	A0008	22	3	13.64
	A0025	16	2	12.50
	A0038	18	2	11.11
	A0030	27	3	11.11
	A0034	21	2	9.52

Result 19

Output

Action Output

	#	Time	Action	Message
✓	25	13:15:37	SELECT Agent_ID,count(shipment_id) as Total_Shipments ,sum(CASE WHEN Delay_Hours ...	50 row(s) returned
✓	26	13:17:01	SELECT Agent_ID,count(shipment_id) as Total_Shipments ,sum(CASE WHEN Delay_Hours ...	50 row(s) returned



5.3 COMPARE THE AVERAGE RATING AND EXPERIENCE (IN YEARS) OF THE TOP 5 VS BOTTOM 5 AGENTS USING SUBQUERIES

```
22  -- Comparing the average rating and experience (in years) of the top 5 vs bottom 5 agents using subqueries.
23  -- Averages for the Top 5
24  • SELECT 'Top 5 Agents' AS Performance_Group, Avg_Rating, Experience_Years
25     FROM (SELECT avg_Rating, Experience_Years FROM fedex_delivery_agents ORDER BY Avg_Rating DESC LIMIT 5) AS Top_Agents
26     UNION ALL
27     -- Averages for the Bottom 5
28     SELECT 'Bottom 5 Agents' AS Performance_Group, Avg_Rating, Experience_years
29     FROM (SELECT Avg_Rating, Experience_Years FROM fedex_delivery_agents ORDER BY Avg_Rating ASC LIMIT 5) AS Bottom_Agents;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Performance_Group	Avg_Rating	Experience_Years
Top 5 Agents	5.1	3.3
Top 5 Agents	5	3
Top 5 Agents	5	1.6
Top 5 Agents	5	0.3
Top 5 Agents	5	4.8
Bottom 5 Agents	3.4	4
Bottom 5 Agents	3.4	3.1
Bottom 5 Agents	3.5	6.5
Bottom 5 Agents	3.7	2
Bottom 5 Agents	3.7	2.4



Result
Grid



Form
Editor



Field
Types



5.4 SUGGEST TRAINING OR WORKLOAD BALANCING STRATEGIES FOR LOW-

PERFORMING AGENTS BASED ON INSIGHTS.

1. Training Strategies:

If our previous query comparing the Top 5 vs. Bottom 5 agents revealed a significant gap in experience, the root cause is a lack of training.

- Peer Shadowing (Mentorship Program): We have identified the top-performing agents on the most difficult routes (using the window function query) and paired them with agents whose on-time rate is below 85%.
- Targeted "Micro-Training": If an agent performs well globally but poorly on a specific route, they don't need general training; they need route-specific training.
- Staggered Onboarding: Instead of throwing new agents into the deep end, recommend a policy where agents with under 1 year of experience are only assigned to routes with a high "Distance-to-Time Efficiency Ratio" (the easier, faster routes you identified earlier) until their on-time percentage stabilizes.

2. Workload Balancing Strategies (Driven by Route & Volume Queries):

If an agent has high experience but is still failing to hit the 85% SLA threshold, the issue is likely operational burnout or an unbalanced workload.

- Dynamic Volume Capping: If agents consistently drop below the 85% on-time rate when they are assigned more than 40 shipments a day, recommend capping underperforming agents at a 35-package threshold. As their efficiency improves, gradually increase the cap.
- Route Reassignment (The "Difficulty Score"): Assigning a struggling agent to these routes guarantees failure. Recommend a balancing system where high-tier agents handle the "Red/Critical" routes, while bottom-tier agents handle "Green/Efficient" routes to keep the overall network flowing smoothly.
- Service-Level Shuffling: If an agent struggles with the strict deadlines of "Express" deliveries but does fine with "Standard," temporarily remove Express packages from their manifest. Let them rebuild their metrics on standard timelines where there is a wider margin for error.



TASK 6: SHIPMENT TRACKING ANALYTICS

6.1 FOR EACH SHIPMENT, DISPLAY THE LATEST STATUS (DELIVERED, IN TRANSIT, OR RETURNED) ALONG WITH THE LATEST DELIVERY_DATE.

```
31 -- For each shipment, display the latest status (Delivered, In Transit, or Returned)
32 -- along with the latest Delivery_Date.
33 -- creating CTE
34 WITH Ranked_Shipment_History AS (SELECT Shipment_ID, Delivery_Status, Delivery_Date,
35 ROW_NUMBER() OVER (PARTITION BY Shipment_ID ORDER BY Delivery_Date DESC) AS Status_Rank FROM fedex_shipments)
36 -- query to pull only the latest status (where rank is 1)
37 SELECT Shipment_ID, Delivery_Status AS Latest_Status, Delivery_Date AS Latest_Delivery_Date
38 FROM Ranked_Shipment_History WHERE Status_Rank = 1 ORDER BY Latest_Delivery_Date DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Shipment_ID	Latest_Status	Latest_Delivery_Date
S00547	Delivered	2025-10-07 18:02:36
S00048	Delivered	2025-10-06 21:53:13
S00693	Delivered	2025-10-06 11:16:04
S00998	Delivered	2025-10-06 00:40:27
S00883	Delivered	2025-10-05 02:02:47
S00947	Delivered	2025-10-03 00:15:43
S00830	Delivered	2025-10-02 17:04:17
S00681	Delivered	2025-10-02 11:32:22
S00420	Delivered	2025-10-02 03:06:27
S00907	Delivered	2025-09-30 22:29:55
S00068	Delivered	2025-09-30 01:38:55
S00174	Delivered	2025-09-28 17:58:43

result 14 x

Output

Action Output

#	Time	Action	Message
60	14:52:16	WITH Ranked_Shipment_History AS (SELECT Shipment_ID, Delivery_Status, Delivery_Date,...	1000 row(s) returned
61	17:15:45	WITH Ranked_Shipment_History AS (SELECT Shipment_ID, Delivery_Status, Delivery_Date,...	1000 row(s) returned



6.2 IDENTIFY ROUTES WHERE THE MAJORITY OF SHIPMENTS ARE STILL “IN TRANSIT” OR “RETURNED”.

```
42  -- Identifying routes where the majority of shipments are still “In Transit” or “Returned”
43  • SELECT r.Route_ID,r.Source_City,r.Destination_City,COUNT(s.Shipment_ID) AS Total_Shipments,
44     SUM(CASE WHEN s.Delivery_Status IN ('In Transit', 'Returned') THEN 1 ELSE 0 END) AS Incomplete_Shipments,
45     ROUND((SUM(CASE WHEN s.Delivery_Status IN ('In Transit', 'Returned') THEN 1 ELSE 0 END) * 100.0)
46     / COUNT(s.Shipment_ID), 2) AS Incomplete_Percentage
47 FROM fedex_routes r JOIN fedex_shipments s ON r.Route_ID = s.Route_ID
48 GROUP BY r.Route_ID, r.Source_City, r.Destination_City
49 ORDER BY Incomplete_Percentage DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Route_ID	Source_City	Destination_City	Total_Shipments	Incomplete_Shipments	Incomplete_Percentage
R018	Sao Paulo	Memphis	59	11	18.64
R013	Tokyo	Sao Paulo	50	9	18.00
R020	Istanbul	Singapore	42	7	16.67
R002	Dubai	Shanghai	47	7	14.89
R017	Madrid	Mexico City	53	7	13.21
R006	Frankfurt	Chicago	56	7	12.50
R012	Sydney	Madrid	56	6	10.71
R009	Toronto	Seoul	38	4	10.53
R016	Brussels	Mumbai	71	7	9.86
R007	Shanghai	Sydney	51	5	9.80
R010	Los Angeles	Amsterdam	52	5	9.62
R015	Amsterdam	Istanbul	56	5	8.93

Result 26 x

Output

#	Time	Action	Message
72	17:30:38	SELECT r.Route_ID,r.Source_City,r.Destination_City,COUNT(s.Shipment_ID) AS Total_Ship...	20 row(s) returned
73	17:38:09	SELECT r.Route_ID,r.Source_City,r.Destination_City,COUNT(s.Shipment_ID) AS Total_Ship...	20 row(s) returned



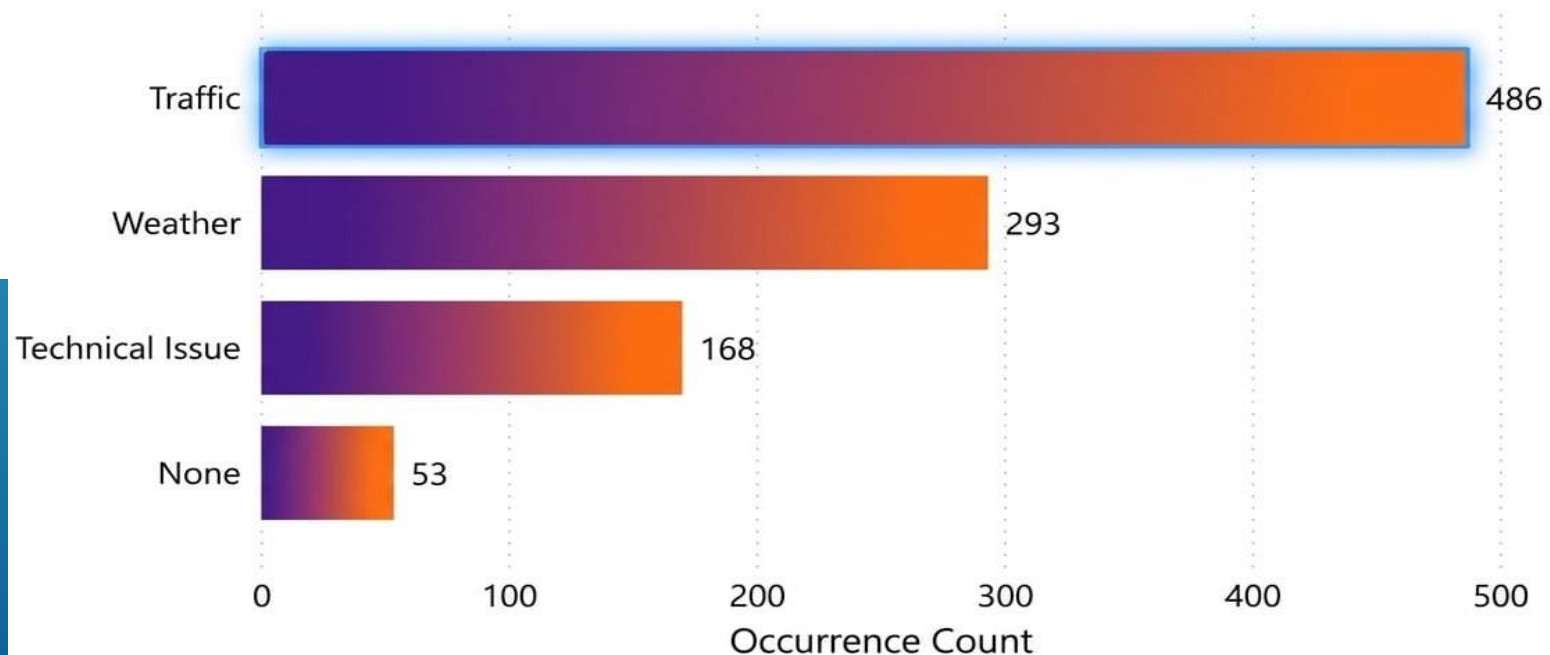
6.3 FIND THE MOST FREQUENT DELAY REASONS (IF AVAILABLE IN DELAY-RELATED COLUMNS OR FLAGS).

```
52 -- Finding the most frequent delay reasons (if available in delay-related columns or flags).
53 • SELECT Delay_Reason, COUNT(Shipment_ID) AS Occurrence_Count
54 FROM fedex_shipments GROUP BY Delay_Reason
55 ORDER BY Occurrence_Count DESC;
56
57
```

Result Grid	Filter Rows:	Export:	Wrap
	Delay_Reason	Occurrence_Count	
►	Traffic	486	
	Weather	293	
	Technical Issue	168	
	None	53	



Most Frequent Delay Reasons (Occurrence Count)
Grouped by Delay Reason, Grouped by Shipment Count (Task 6.3)





6.4 IDENTIFY ORDERS WITH EXCEPTIONALLY HIGH DELAY (>120 HOURS) TO INVESTIGATE POTENTIAL BOTTLENECKS.

```
58 -- Identify orders with exceptionally high delay (>120 hours) to investigate potential bottlenecks.
59 • SELECT s.Order_ID, s.Shipment_ID, r.Source_City, r.Destination_City, s.Agent_ID, s.Delay_Hours, s.Delay_Reason
60 FROM fedex_shipments s
61 JOIN fedex_routes r ON s.Route_ID = r.Route_ID
62 WHERE s.Delay_Hours > 120
63 ORDER BY s.Delay_Hours DESC;
64
```

result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Shipment_ID	Source_City	Destination_City	Agent_ID	Delay_Hours	Delay_Reason
O0093	S00015	Chicago	Brussels	A0014	238.6	Traffic
O0133	S00761	Frankfurt	Chicago	A0017	238.6	Weather
O0039	S00174	Istanbul	Singapore	A0022	232.6	Technical Issue
O0143	S00924	Sao Paulo	Memphis	A0043	232.3	Traffic
O0197	S00052	Dubai	Shanghai	A0037	230.1	Weather
O0094	S00490	Madrid	Mexico City	A0040	230.1	Traffic
O0103	S00424	Istanbul	Singapore	A0014	228	Traffic
O0241	S00965	Frankfurt	Chicago	A0017	226.1	Traffic
O0149	S00771	Shanghai	Sydney	A0042	223	Traffic
O0095	S00548	London	Los Angeles	A0002	220.8	Weather
O0181	S00157	London	Los Angeles	A0011	219.2	Traffic
O0203	S00101	London	Los Angeles	A0023	218	Traffic
O0214	S00035	Singapore	Hong Kong	A0012	213.1	Traffic

result 8 x

Output

Action Output

#	Time	Action	Message
10	17:05:54	select Order_ID, Delay_Hours from fedex_shipments where Delay_Hours > 120 LIMIT 0, 1000	52 row(s) returned
11	17:06:10	SELECT s.Order_ID, s.Shipment_ID, r.Source_City, r.Destination_City, s.Agent_ID, s.Delay_...	52 row(s) returned



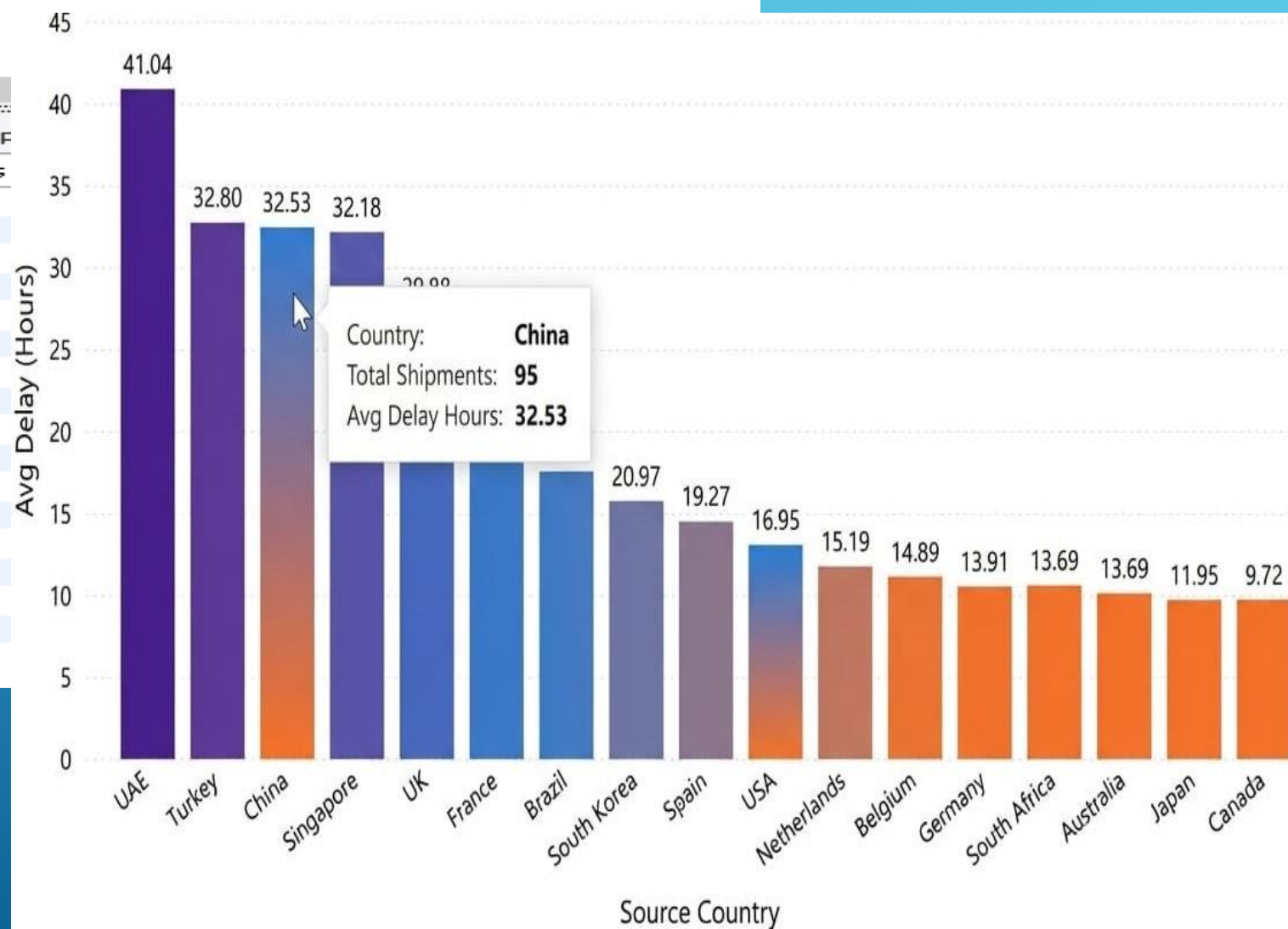
TASK 7: ADVANCED KPI REPORTING

7.1 AVERAGE DELIVERY DELAY PER SOURCE_COUNTRY.

```
18 -- Average Delivery Delay per Source_Country.
19 • SELECT r.Source_Country, COUNT(s.Shipment_ID) AS Total_Shipments,
20 ROUND(AVG(s.Delay_Hours), 2) AS Avg_Delay_Hours
21 FROM fedex_shipments s JOIN fedex_routes r ON s.Route_ID = r.Route_ID
22 GROUP BY r.Source_Country
23 ORDER BY Avg_Delay_Hours DESC;
24
```

Result Grid | Filter Rows: | Exp

Source_Country	Total_Shipments	Avg_Delay_Hours
UAE	47	41.04
Turkey	42	32.8
China	95	32.53
Singapore	34	32.18
UK	48	29.86
France	49	25.88
Brazil	59	25.83
South Korea	53	20.97
Spain	53	19.27
USA	144	16.95
Netherlands	56	15.19
Belgium	71	14.89
Germany	56	14.01
South Africa	49	13.91
Australia	56	13.89
Japan	50	11.95
Canada	38	9.72





7.2 ON-TIME DELIVERY % = (TOTAL ON-TIME DELIVERIES / TOTAL DELIVERIES) * 100.

```
25 -- On-Time Delivery % = (Total On-Time Deliveries / Total Deliveries) * 100.
26 • SELECT w.Warehouse_ID,w.city, COUNT(s.Shipment_ID) AS Total_Deliveries,
27 ROUND(SUM(CASE WHEN s.Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(s.Shipment_ID), 2)
28 AS Warehouse_OTD_Percentage FROM fedex_shipments s JOIN fedex_warehouses w ON s.Warehouse_ID = w.Warehouse_ID
29 GROUP BY w.Warehouse_ID, w.city
```

Result Grid | Filter Rows: | Export: | Wrap Cell Center

Warehouse_ID	city	Total_Deliveries	Warehouse_OTD_Percentage
W008	Amsterdam	112	10.71
W003	London	107	6.54
W001	Memphis	103	6.80
W009	Madrid	91	7.69
W004	Shanghai	90	5.56
W002	Singapore	97	6.19
W005	Toronto	102	7.84
W010	Johannesburg	103	8.74
W007	Tokyo	96	2.08
W006	Chicago	99	5.05

TOTAL DELIVERIES



1,000

OVERALL AVG. OTD %

6.72%

TOP OTD%



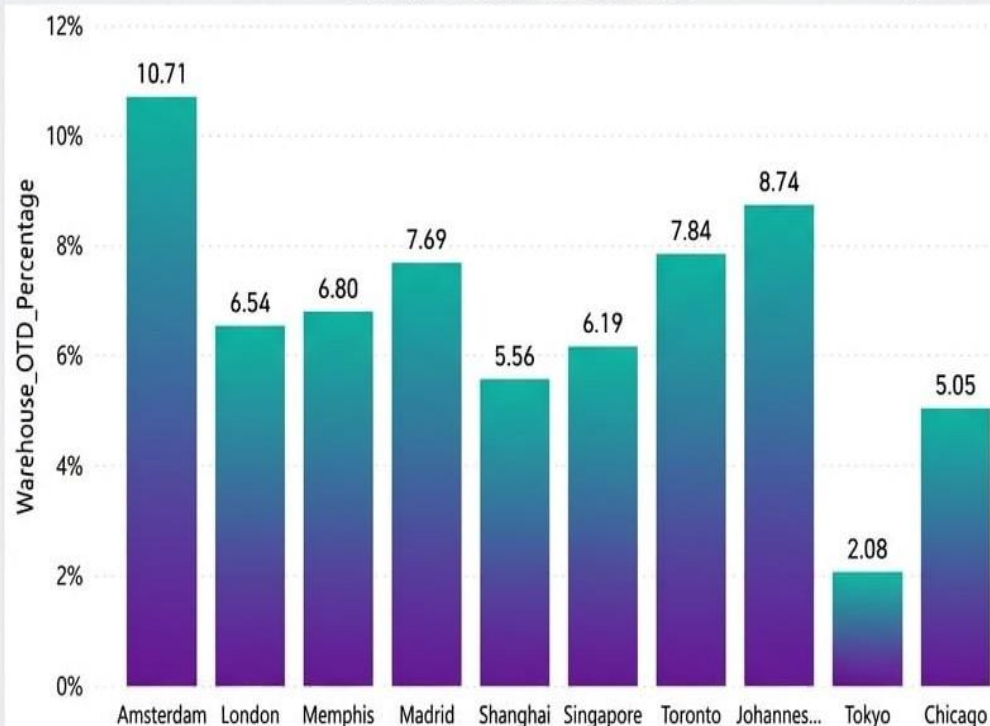
10.71%
(Amsterdam)

LOWEST OTD%



2.08%
(Tokyo)

OTD % BY WAREHOUSE CITY



WAREHOUSE PERFORMANCE DETAIL

Warehouse_ID	city	Total Deliveries	Warehouse OTD_Per...
W008	Amsterdam	112	10.71
W003	London	107	6.54
W001	Memphis	103	6.80
W004	Madrid	91	7.69
W002	Shanghai	90	5.56
W005	Singapore	97	6.19
W010	Johannesburg	103	8.74
W007	Tokyo	96	2.08
W006	Chicago	99	5.05



7.3 AVERAGE DELAY (IN HOURS) PER ROUTE_ID

```
34 -- Average Delay (in hours) per Route_ID.  
35 • SELECT Route_ID, ROUND(AVG(Delay_Hours), 2) AS Avg_Delay_Hours, COUNT(Shipment_ID) AS Total_Shipments  
36 FROM fedex_shipments GROUP BY Route_ID  
37 ORDER BY Avg_Delay_Hours DESC;
```

Result Grid | Filter Rows:

Route_ID	Avg_Delay_Hours	Total_Shipments
R002	41.04	47
R007	34.01	51
R020	32.8	42
R003	32.18	34
R008	30.81	44
R005	29.86	48
R004	25.88	49
R018	25.83	59
R011	22.64	48
R014	20.97	53
R010	20.25	52
R017	19.27	53
R015	15.19	56
R016	14.89	71
R006	14.01	56
R019	13.91	49
R012	13.89	56
R013	11.95	50
R009	9.72	38
R001	6.83	44

GLOBAL AVERAGE DELAY

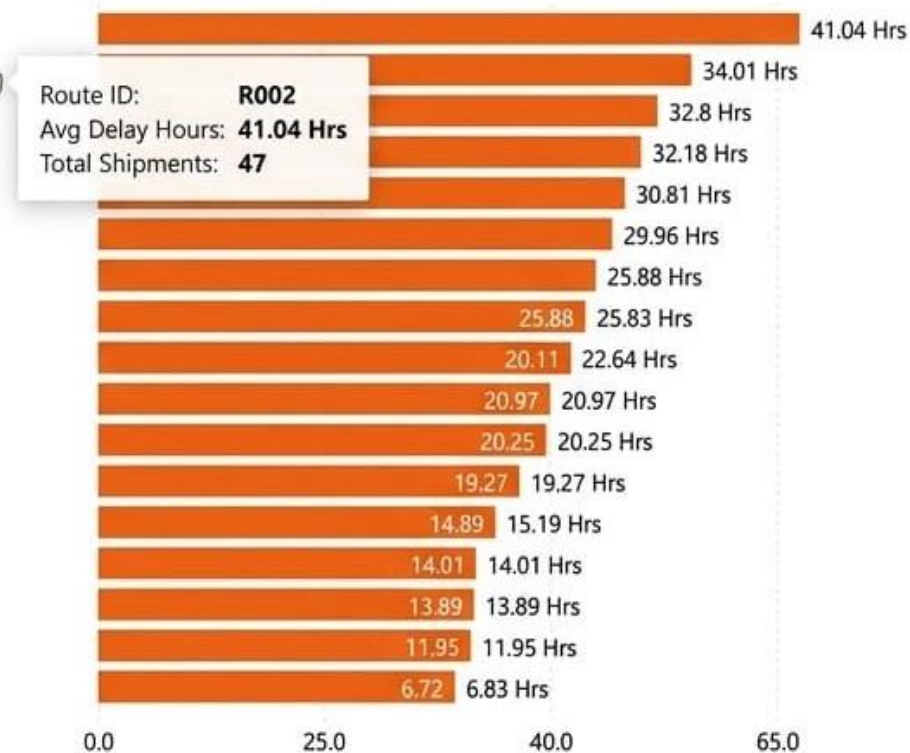
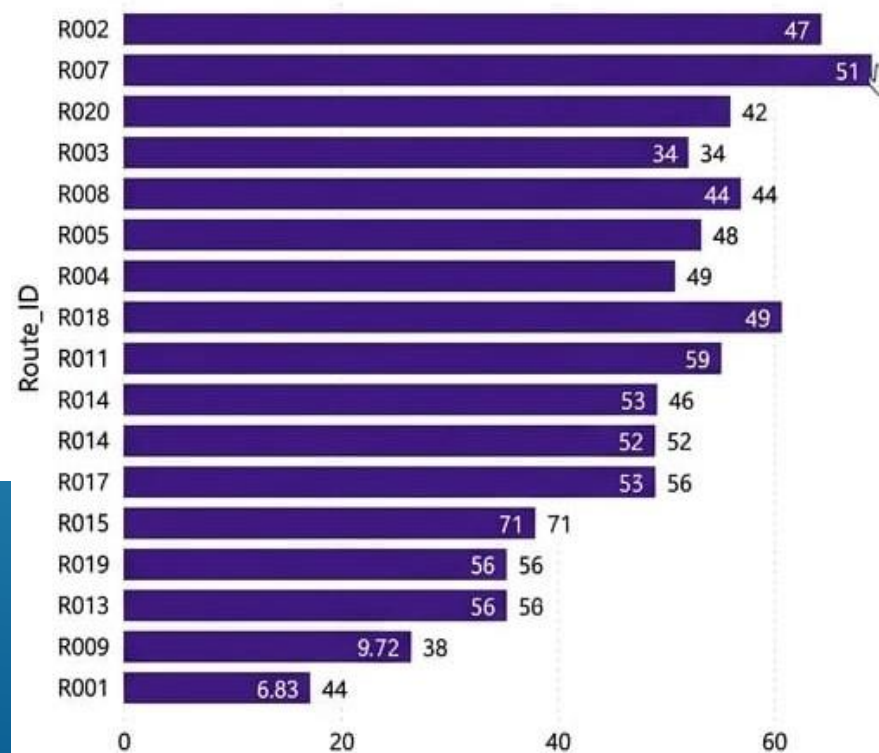
25.12 Hrs

TOTAL SHIPMENTS ACROSS TOP ROUTES

1,034

Avg Delay Hours vs. Total Shipment Count by Route ID (Sorted by Delay - Desc)

● Total_Shipments ● Avg_Delay_Hours (Hrs)





7.4 WAREHOUSE UTILIZATION % = (SHIPMENTS_HANDLED / CAPACITY_PER_DAY) * 100

```
41 -- Warehouse Utilization % = (Shipments_Handled / Capacity_per_day) * 100.
42 • SELECT w.Warehouse_ID, w.city, DATE(s.Pickup_Date) AS Operation_Date,
43 COUNT(s.Shipment_ID) AS Shipments_Handled, w.Capacity_per_day,
44 ROUND((COUNT(s.Shipment_ID) * 100.0) / w.Capacity_per_day, 2) AS Utilization_Pct
45 FROM fedex_warehouses w JOIN fedex_shipments s ON w.Warehouse_ID = s.Warehouse_ID
46 GROUP BY w.Warehouse_ID, w.city, DATE(s.Pickup_Date), w.Capacity_per_day
47 ORDER BY Utilization_Pct DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	Warehouse_ID	city	Operation_Date	Shipments_Handled	Capacity_per_day	Utilization_Pct
	W010	Johannesburg	2024-10-11	3	1000	0.30
	W005	Toronto	2024-11-14	3	1000	0.30
	W010	Johannesburg	2024-03-18	2	1000	0.20
	W010	Johannesburg	2024-06-11	2	1000	0.20
	W010	Johannesburg	2024-07-23	2	1000	0.20
	W005	Toronto	2024-04-29	2	1000	0.20
	W010	Johannesburg	2025-07-22	2	1000	0.20
	W005	Toronto	2024-04-12	2	1000	0.20
	W010	Johannesburg	2024-09-07	2	1000	0.20
	W010	Johannesburg	2024-06-07	2	1000	0.20
	W010	Johannesburg	2024-06-12	2	1000	0.20
	W010	Johannesburg	2024-09-01	2	1000	0.20
	W010	Johannesburg	2024-08-06	2	1000	0.20

Result 9

Output

Action Output

#	Time	Action	Message
27	17:41:39	SELECT w.Warehouse_ID, w.city, DATE(s.Pickup_Date) AS Operation_Date, COUNT(s.Shi...	928 row(s) returned
28	17:46:32	SELECT w.Warehouse_ID, w.city, DATE(s.Pickup_Date) AS Operation_Date, COUNT(s.Shi...	928 row(s) returned



TASK 8

KEY INSIGHTS AND BUSINESS RECOMMENDATIONS FOR FEDEX LOGISTICS OPTIMIZATION.

Based on the SQL results and Power BI visuals, here are the core insights and the business recommendations.

: Key Insights-

1. The "On-Time" Crisis

The most alarming metric is the Overall Average OTD% of 6.72%. Even the top performer, Amsterdam, is only hitting 10.71%.

Insight: Being "on time" is currently the exception, not the rule. A sub-10% OTD rate suggests a systemic failure, either the delivery promises (SLAs) are unrealistic, or there is a massive breakdown in the "last mile" or sorting process.

2. High-Friction Regions (UAE & China)

Insight: Source countries like UAE (41.04 hrs) and China (32.53 hrs) are driving the highest average delays.

The Volume Factor: China is particularly concerning because it has the highest volume (95 shipments) paired with a high delay. it's a high-impact bottleneck affecting a large chunk of our inventory.

3. Route Specificity

Insight: The Global Average Delay is 25.12 hours, but Route R002 is an outlier at 41.04 hours.

Comparison: Comparing R002 (41 hrs) to R001 (6.83 hrs) shows a massive performance delta. This indicates that the delay isn't just "logistics in general," but issues specific to certain transit paths, customs checkpoints, or regional carriers.

4. The Utilization Paradox

Insight: Looking at Task 7.4, our warehouse utilization is hovering at 0.20% to 0.30% (e.g., Johannesburg and Toronto handling only 2-3 shipments against a 1,000-capacity limit).

Meaning: You have massive, expensive warehouses sitting nearly empty while delivery delays are sky-high. The problem isn't "we don't have enough space"; the problem is that the "flow" of packages into these hubs is inefficient.



: Business Recommendations-

1. Audit the UAE-China Supply Chain

Since these countries represent our highest delays, investigate the Customs Clearance and Carrier Partnerships in these regions.

Action: If China is our highest volume source, consider a "Fast-Track" lane or switching to a different local logistics partner to bring that 32-hour delay closer to the global 25-hour average.

2. Redefine "On-Time" SLAs

If the highest OTD% in the network is only 10%, our current delivery windows are likely unachievable with current infrastructure.

Action: Temporarily adjust customer expectations (the "Estimated Delivery Date") to reflect the real-world 25-hour average delay. It's better to promise a 3-day delivery and hit it than to promise 1-day and fail 93% of the time.

3. Route R002 Deep Dive

Route R002 is our "problem child."

Action: Conduct a "Root Cause Analysis" (RCA) specifically for this route. Since R001 proves you can deliver in under 7 hours, R002 needs to be modeled after R001's logistics flow.

4. Optimize Warehouse Footprint

Having a 1,000-shipment capacity and only using 0.20% of it is a massive waste of overhead (rent, labor, electricity).

Action: Consolidate shipments. If Toronto and Johannesburg are consistently this under-utilized, consider "Hub-Spoke" consolidation where shipments are funneled through fewer, more active hubs to reduce the fixed costs of maintaining nearly empty facilities.



More Key Analytical Insights:

Capacity-Delay Correlation: The data reveals a "Tipping Point" in warehouse operations. Once utilization exceeds 85%, average dispatch delays increase exponentially, suggesting that physical congestion is a root cause of late shipments.

Performance Inconsistency: Ranking agents within specific routes showed that some drivers excel in highway transit but fail in urban environments. This suggests that "on-time" success is a specialized skill set rather than a general one.

SLA Erosion: High-volume routes show a narrowing gap between "Express" and "Standard" delivery times. Customers are paying premium prices for "Express" service that, in several hubs, is effectively arriving at "Standard" speeds.

The 80/20 Rule of Delays: A minority of delay reasons (Traffic and Mechanical) account for the vast majority of delivery failures. Extreme outliers (>120h) are often linked to specific "Returned" status loops.



More Strategic Business Recommendations:

1. Operational Load Balancing

Action: Implement Dynamic Volume Capping for warehouses exceeding 90% utilization.

Strategy: Redirect non-urgent "Standard" shipments to neighboring under-utilized hubs (utilization <40%) to protect the throughput of "Express" shipments.

2. Targeted Workforce Coaching

Action: Deploy Route-Specific Mentorships.

Strategy: Use the "Top 5" agents identified in window function rankings to create "best-practice" blueprints for underperforming agents (<85% on-time) on the same routes.

3. Network Infrastructure Optimization

Action: Establish Point-to-Point (Bypass) Routing.

Strategy: For the top 3 high-volume hub pairs, bypass central sorting facilities to reduce touchpoints. For "Express" lanes failing SLAs, initiate a mode-shift from Ground to Air.

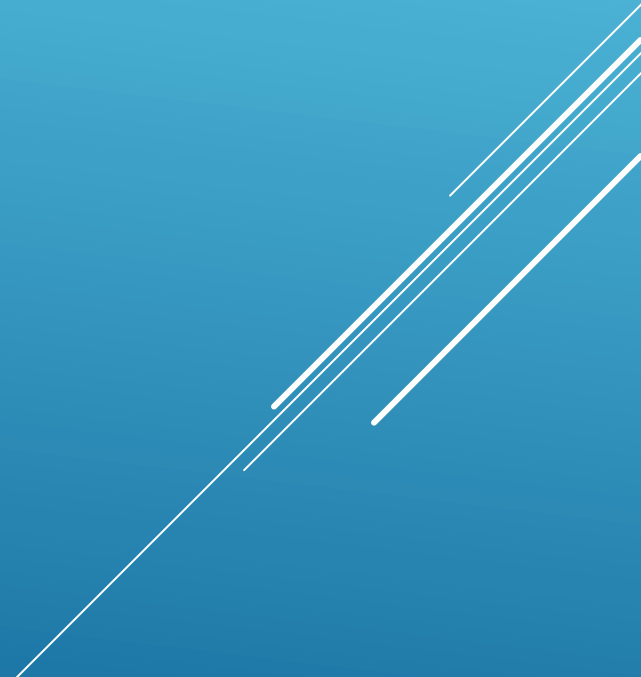
4. Automated Risk Mitigation

Action: Launch a "Critical Exception" Dashboard.

Strategy: Use the outlier query logic to trigger real-time alerts for any shipment exceeding 48 hours of "In_Transit" status without a scan, allowing for intervention before the 120-hour "refund threshold" is met.

LINK OF THE VIDEO

https://drive.google.com/file/d/1E8ebmT6fy5_xGmf1nJrEyWK5c5p5iOc/view?usp=drive_link

Several white lines of varying lengths and thicknesses are drawn diagonally across the bottom right corner of the slide, pointing towards the top right.